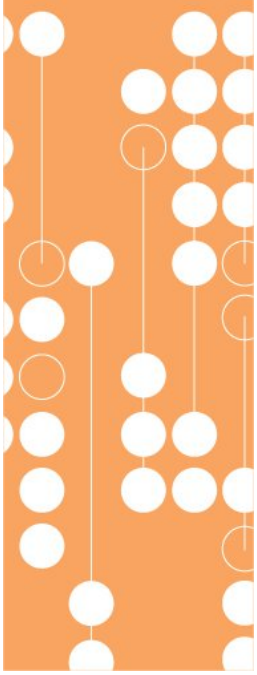
Advanced ReactJS

Advanced React Labs and Challenges



Contents

1. **Quick Lab 1 – useReducer and useRef** 1
   1.1. Objectives 1
   1.2. Activity 1
2. Quick Lab 2 – **useMemo and useCallback** 3
   2.1. Objectives 3
   2.2. Activity 3
   2.3. Objectives 4
   2.4. Activity 4
3. Quick Lab 4 – **Redux** 7
   3.1. Objectives 7
   3.2. Activity 7

1. Quick Lab 1 – useReducer and useRef

1.1. Objectives

- To be able to create a managed form in React using useReducer and useRef

1.2. Activity

1. Open the lab 1 starter in VSCode.
2. Install dependencies.
3. Create an **inputRef** using **useRef** and add it to the **addTask** input
4. In TaskManager.jsx initialise a new reducer with useReducer
5. Update the **reducer** function with three action types: **ADD_TASK**, **TOGGLE_TASK** AND **CLEAR_COMPLETED**. At the moment these can just return a blank object and remember to add a default case to the switch statement that just returns the **state** object as-is.
6. In the **handleTask** function use the **inputRef** to grab the value of the **addTask** field then dispatch an event with **type: "ADD_TASK"** and **payload:value**
7. Clear the input.
8. Focus the input again.
9. Update the **ADD_TASK** case**:**

```
case "ADD_TASK":
  return {
    ...state,
    tasks: [...state.tasks, { id: Date.now(), text: action.payload, done: false }]
  };
```

10. Add the **handleTask** function as the **onSubmit** for the form
11. Update the **TOGGLE_TASK** case**:**

```
case "TOGGLE_TASK":
  return {
    ...state,
    tasks: state.tasks.map(t =>
      t.id === action.payload ? { ...t, done: !t.done } : t
    )
  };
```

12. Call **dispatch** from the **onClick** of the **li** with **type: "TOGGLE_TASK"** and **payload: task.id**

13. Update the **CLEAR_COMPLETED** case:

```
    }
case "CLEAR_COMPLETED":
    return { ...state, tasks: state.tasks.filter(task => !task.done) };
default:
    return state;
}
```

14. Call dispatch from the **onClick** of the clear button with **type: “CLEAR_COMPLETED**”

2. Quick Lab 2 – **useMemo and useCallback**

2.1. Objectives

- To be able to use **useMemo** and useCallbck to make a React app more efficient

2.2. Activity

1. Open the lab 1 starter in VSCode.
2. Open **SearchList.jsx**
3. Use **useCallback** to memoise the handleSelect function to prevent unnecessary re-renders
4. **Use useMemo** to memoise the item filtering (makng sure to add a **query** dependency) this will ensure the filtering operation doesn't slow down the application by being run unnecessarily.

Quick Lab 3 – **useActionState, useOptimistic and useFormStatus**

2.3. Objectives

- To be able to use **useActionState**, **useOptimistic and useFormStatus** in a React app

2.4. Activity

1. Open the lab 1 starter in VSCode.
2. Open **TodoApp.jsx**
3. Add the optimistic state to **ToDoApp**

```
export default function TodoApp({ initialTodos }) {
  // TODO: 1. Add optimistic state
  const [optimisticTodos, updateOptimisticTodos] = useOptimistic(
    initialTodos,
    (current, action) => {
      switch (action.type) {
        case "add":
          return [...current, action.todo];
        case "toggle":
          return current.map(t =>
            t.id === action.id ? { ...t, completed: !t.completed } : t
          );
        case "delete":
          return current.filter(t => t.id !== action.id);
        default:
          return current;
      }
    }
  );
```

4. Add the **useActionState** for **addTodo**

```
  // TODO: 4. useActionState for addTodo
  const [state, formAction] = useActionState(async (prev, formData) => {
      const todo = await addTodo(formData);
      updateOptimisticTodos({ type: "add", todo });
      return todo;
  }, null);
```

15. Add the **formAction** to the **AddTodoForm** via the **action** prop

```
<div style={{ maxWidth: 400, margin: "2rem auto
    <h2>Todo List</h2>

    <AddTodoForm action={formAction} />

    <ul>
        {optimisticTodos.map(todo => (
```

16. Add the **formAction**s to toggle and delete todos

```
        {todo.text}
    </span>
    {/* TODO 14. Add the formActions to toggle and delete todos */}
    <button
        formAction={async () => {
            updateOptimisticTodos({ type: "toggle", id: todo.id });
            await toggleTodo(todo.id);
        }}
    >
        Toggle
    </button>

    <button
        formAction={async () => {
            updateOptimisticTodos({ type: "delete", id: todo.id });
            await deleteTodo(todo.id);
        }}
    >
        Delete
    </button>
```

17. Add **useFormStatus** to **SubmitButton** – this will allow the submit button to be disabled whilst the submit is in progress

```
function SubmitButton() {
    // TODO: add useFormStatus to SubmitButton
    const { pending } = useFormStatus();

    return (
        <button type="submit" disabled={pending}>
            {pending ? "Adding..." : "Add"}
        </button>
    );
}
```

3. Quick Lab 4 – **Redux**

3.1. Objectives

- To implement React Redux to manage state in a simple application.

3.2. Activity

1. Open the starter **02-redux** in VSCode
2. Install dependencies
3. Open **store.js**; this is where we will be creating the store for this React app. Start by creating a slice called **booksSlice** with **name: "books"**, an **initialState: { items: [], filter: "all"** and **reducers: {}**

```
const booksSlice = createSlice({
  name: "books",
  initialState: {
    items: [],
    filter: "all" // all | read | unread
  },
  reducers: {}
});
```

4. Add reducers **addBook**, **removeBook**, **toggleRead** and **setFilter**

```
reducers: {
  addBook(state, action) {
    state.items.push({
      id: Date.now(),
      title: action.payload,
      read: false
    });
  },
  removeBook(state, action) {
    state.items = state.items.filter(b => b.id !== action.payload);
  },
  toggleRead(state, action) {
    const book = state.items.find(b => b.id === action.payload);
    if (book) book.read = !book.read;
  },
  setFilter(state, action) {
    state.filter = action.payload;
  }
}
```

5. Export the actions from **bookSlice**

```
// TODO 5. Export actions from booksSlice
export const { addBook, removeBook, toggleRead, setFilter } = booksSlice.actions;
```

6. Configure and export the store

```
// TODO 4. Configure and export the store
export const demoStore = configureStore({
  reducer: {
    books: booksSlice.reducer
  }
});
```

7. In order to access the store in **App** it must be provided so next add a **provider** wrapped around **<App/>** in **main.jsx**, making sure to import the demoStore you just created.

```
// TODO 7. Setup Provider around App, importing the demoStore
createRoot(document.getElementById('root')).render(
  <Provider store={demoStore}>
    <StrictMode>
      <App />
    </StrictMode>
  </Provider>
);
```

8. Fetch **dispatch** function from the store in **App.js**

```
export default function App() {
  // TODO 6. Fetch dispatch function from the store
  const dispatch = useDispatch();
```

9. Fetch items and filter from the store

```
// TODO 7. Fetch items and filter from the store
const { items, filter } = useSelector(state => state.books);
```

10. Filter the **items** array using the **filter** you just extracted from the store

```
// TODO 10. Filter items
const filteredBooks = items.filter(book => {
  if (filter === "read") return book.read;
  if (filter === "unread") return !book.read;
  return true;
});
```

11. Use dispatch to update the filter value when the corresponding button is clicked

```
<div style={{ marginTop: "1rem" }}>
  {/* TODO 11. Use dispatch to update the filter value when the corresponding button is clicked */}
  <button onClick={() => dispatch(setFilter("all"))} disabled={filter === "all"}>All</button>
  <button onClick={() => dispatch(setFilter("read"))} disabled={filter === "read"}>Read</button>
  <button onClick={() => dispatch(setFilter("unread"))} disabled={filter === "unread"}>Unread</button>
</div>
```

12. Use **dispatch** to toggle and remove books

```
    </span>
    {/* TODO 12. use dispatch to toggle and remove books */}
    <button onClick={() => dispatch(toggleRead(book.id))}>
      {book.read ? "Mark Unread" : "Mark Read"}
    </button>

    <button onClick={() => dispatch(removeBook(book.id))}>
      Remove
    </button>
  </li>
```

13. Update the **handleSubmit** function to add a book using **dispatch** – remember to blank the **title** afterwards.

```
// TODO 13. Create the handleSubmit
function handleSubmit(e) {
  e.preventDefault();
  if (!title.trim()) return;
  dispatch(addBook(title));
  setTitle("");
}
```

14. Test your App in the browser

Learn. To Change.

QA.com